Proyecto 2

Indicaciones:

- *Los estudiantes deberán trabajar en grupos y desarrollar uno de los proyectos propuestos en el curso.*
- *Cada proyecto deberá ser implementado de manera funcional, evidenciando la correcta aplicación de los conceptos de compiladores y lenguajes de programación desarrollados durante el curso.*
- *El desarrollo deberá incluir documentación técnica clara y estructurada, describiendo la arquitectura del sistema, las decisiones de diseño, las tecnologías utilizadas y los principales componentes implementados.*
- *Cada grupo deberá realizar una exposición técnica del proyecto, demostrando el funcionamiento del sistema y explicando adecuadamente los componentes desarrollados, las decisiones de implementación y los resultados obtenidos.*
- *Se recomienda el uso de herramientas modernas y buenas prácticas de ingeniería de software para la organización, implementación y documentación del proyecto.*
- *Todos los integrantes del equipo deberán participar activamente en el desarrollo del proyecto, demostrando contribuciones reales, verificables y consistentes con el trabajo presentado.*
- *Todos los integrantes del grupo deberán demostrar capacidad de análisis, investigación, implementación y comprensión profunda de los conceptos de compiladores y lenguajes de programación involucrados en el proyecto.*
- *El uso de herramientas de inteligencia artificial estará permitido únicamente como apoyo complementario durante el desarrollo del proyecto. Sin embargo, el trabajo deberá reflejar comprensión técnica real por parte de todos los miembros del grupo. En caso de evidenciarse dependencia excesiva de herramientas de IA, incapacidad para explicar el funcionamiento del código presentado o desconocimiento de los componentes implementados por parte de uno o más integrantes, el proyecto será calificado con cero (0).*
- *En caso de detectarse que uno o más integrantes no han participado de manera efectiva en el desarrollo del trabajo, el proyecto será calificado con cero (0).*
- *Cada tipo de proyecto cuenta con una actividad bonificadora, correspondiente a su máximo logro, cuyos requisitos se describen en la sección Bonificación de cada proyecto. Los grupos que la completen satisfactoriamente podrán obtener hasta 3 puntos adicionales en el Examen final del curso. Esta actividad es opcional y no forma parte de los objetivos mínimos del proyecto.*
- *Todo grupo que aspire a obtener la bonificación deberá asistir obligatoriamente a tres reuniones de seguimiento programadas: dos con Leonardo (ACL del curso) y una con el profesor. En cada reunión, el grupo deberá presentar los avances de su proyecto, con el objetivo de recibir orientación y retroalimentación constructiva durante el proceso. Los grupos que no cumplan con las tres reuniones no podrán acceder a la bonificación, aun cuando hayan alcanzado el máximo logro del proyecto.*

Tipo 2: Implementación de un compilador con optimización y seguridad

El objetivo del presente proyecto es diseñar e implementar un compilador completo que replique un lenguaje de programación existente sobre arquitectura x86, integrando las etapas fundamentales del proceso de compilación, desde el análisis léxico y sintáctico hasta la generación de código ensamblador. El compilador deberá incorporar, además, un conjunto de técnicas de **optimización** de código y de **seguridad**, orientadas a prevenir, detectar o mitigar errores de memoria y comportamientos no definidos en los programas compilados.

Objetivos

Los estudiantes deberán:

- Seleccionar un lenguaje de programación existente, con especificación o documentación formal disponible (por ejemplo, C, un subconjunto de C++, Rust, Go, Pascal u otro aprobado por el docente). El objetivo es **replicar el lenguaje seleccionado**, por lo que el compilador deberá respetar fielmente sus reglas léxicas, sintácticas y semánticas, su sistema de tipos y su comportamiento en tiempo de ejecución, tal como están definidos en su especificación. Un programa válido en el lenguaje original deberá compilarse y producir el mismo resultado con el compilador desarrollado.
- Definir y documentar el alcance del subconjunto del lenguaje a implementar, justificando las características incluidas y excluidas, y citando las secciones correspondientes de la especificación oficial.
- Implementar un compilador completo que incluya las fases de análisis léxico, sintáctico y semántico.
- Implementar un sistema de verificación de tipos y manejo de errores léxicos, sintácticos y semánticos, con mensajes de diagnóstico claros que indiquen la ubicación del error.
- Diseñar una representación intermedia (por ejemplo, código de tres direcciones o forma SSA) sobre la cual se apliquen los análisis y las optimizaciones.
- Implementar técnicas de optimización sobre la representación intermedia y sobre el código generado (ver sección *Optimización*).
- Implementar mecanismos de seguridad en tiempo de compilación y/o en tiempo de ejecución (ver sección *Seguridad*).
- Generar código ensamblador para arquitectura x86 (32 o 64 bits), ensamblable y enlazable con herramientas estándar.
- Desarrollar un conjunto de *benchmarks* y de programas de prueba para evaluar el rendimiento y la efectividad de los mecanismos de seguridad.
- Realizar una comparación experimental con compiladores de uso extendido como GCC, Clang/LLVM, MSVC, Rustc o Go Compiler.
- Analizar y documentar los resultados obtenidos mediante tablas, gráficos y discusión técnica.
- Presentar y defender el proyecto mediante una exposición técnica.

Optimización

El compilador deberá implementar **al menos cuatro** técnicas de optimización, de las cuales al menos una debe ser no local (global, de bucles o interprocedural). Cada optimización deberá poder activarse o desactivarse mediante opciones del compilador (por ejemplo, `-O0` y `-O1`), de modo que su impacto pueda medirse de forma independiente. Ninguna optimización deberá alterar el comportamiento observable del programa definido por la especificación del lenguaje.

- **Optimizaciones locales:** plegado de constantes (*constant folding*), propagación de constantes, simplificación algebraica, reducción de fuerza (*strength reduction*) y optimización de mirilla (*peephole*) sobre el ensamblador generado.
- **Optimizaciones globales:** eliminación de código muerto, eliminación de subexpresiones comunes, propagación de copias y eliminación de código inalcanzable, basadas en análisis de flujo de datos sobre el grafo de flujo de control.
- **Optimizaciones de bucles:** movimiento de código invariante (*loop-invariant code motion*), desenrollado de bucles (*loop unrolling*) y optimización de variables de inducción.
- **Optimizaciones de backend:** asignación de registros (por ejemplo, *linear scan* o coloreo de grafos) y selección de instrucciones eficiente para x86.
- **Optimizaciones interprocedurales:** expansión en línea de funciones (*inlining*) y optimización de llamadas en cola (*tail call optimization*).

Seguridad

El compilador deberá implementar **al menos tres** mecanismos de seguridad, de los cuales al menos uno debe ser estático y al menos uno dinámico. Los mecanismos dinámicos deberán poder activarse o desactivarse mediante opciones del compilador, con el fin de medir su costo en rendimiento. En caso de que el lenguaje seleccionado ya defina alguno de estos mecanismos en su especificación (por ejemplo, la verificación de límites en Rust o Go), su implementación será obligatoria y deberá seguir el comportamiento establecido por el lenguaje.

- **Análisis estáticos (tiempo de compilación):** detección de variables usadas sin inicializar, funciones sin valor de retorno en todos sus caminos, desreferencias de punteros nulos evidentes, conversiones de tipo inseguras, desbordamiento en expresiones constantes, accesos fuera de rango con índices conocidos y detección de punteros colgantes (*dangling pointers*) mediante análisis de tiempo de vida.
- **Verificaciones dinámicas (tiempo de ejecución):** verificación de límites en accesos a arreglos y cadenas (*bounds checking*), verificación de punteros nulos, detección de desbordamiento aritmético en enteros, división entre cero y límite de profundidad de recursión.
- **Protección de memoria:** canarios de pila (*stack canaries*) para detectar desbordamientos de búfer, detección de doble liberación (*double free*) y de uso después de liberar (*use-after-free*) mediante un asignador de memoria instrumentado.
- **Endurecimiento del binario:** generación de código independiente de la posición (PIE), pila no ejecutable e integridad básica del flujo de control (por ejemplo, validación de direcciones de retorno mediante una pila sombra).

Ante una violación detectada en tiempo de ejecución, el programa deberá terminar de manera controlada, mostrando un mensaje que indique el tipo de error y, de ser posible, su ubicación en el código fuente.

Evaluación experimental

La evaluación deberá considerar, como mínimo, los siguientes aspectos:

- **Fidelidad al lenguaje:** un conjunto de programas de prueba escritos en el lenguaje seleccionado, verificando que el compilador desarrollado produzca los mismos resultados que el compilador de referencia del lenguaje.
- **Rendimiento:** tiempo de ejecución de los *benchmarks* con y sin optimizaciones, comparado con los compiladores de referencia en distintos niveles de optimización (por ejemplo, `-O0` y `-O2`).
- **Calidad del código:** tamaño del binario, número de instrucciones generadas y tiempo de compilación.
- **Impacto de cada optimización:** mejora individual de cada técnica implementada sobre el conjunto de *benchmarks*.
- **Efectividad de la seguridad:** un conjunto de programas con vulnerabilidades intencionales (desbordamientos, accesos fuera de rango, punteros nulos, entre otros), indicando cuáles son detectadas por el compilador desarrollado y cuáles por los compiladores de referencia con sus mecanismos equivalentes, por ejemplo:

  `-fstack-protector` o `-fsanitize=address,undefined`

- **Costo de la seguridad:** sobrecarga en tiempo de ejecución y tamaño del binario introducida por los mecanismos dinámicos.

Las mediciones deberán garantizar su validez estadística mediante múltiples ejecuciones, medidas de dispersión y la descripción detallada del entorno experimental.

Entregables

Al finalizar el proyecto, cada grupo deberá presentar los siguientes entregables en una única entrega:

- **Código fuente:** repositorio con el código del compilador, el conjunto de pruebas, los *benchmarks*, los programas con vulnerabilidades intencionales y las instrucciones para compilar el compilador, ejecutarlo y replicar los experimentos.
- **Reporte técnico:** documento en LaTeX que incluya, como mínimo, las siguientes secciones:
  - **Introducción:** objetivos del proyecto y descripción general del compilador desarrollado.
  - **Lenguaje seleccionado:** descripción del lenguaje, alcance del subconjunto implementado y características excluidas, con referencias a la especificación oficial.

  - **Arquitectura del compilador:** descripción de sus fases y componentes, acompañada de diagramas del flujo de compilación.
  - **Frontend:** análisis léxico, sintáctico y semántico, sistema de tipos y manejo de errores.
  - **Representación intermedia:** diseño de la IR y su construcción a partir del AST.
  - **Optimización:** descripción de cada técnica implementada, con ejemplos del código antes y después de aplicarla.
  - **Seguridad:** descripción de cada mecanismo implementado, con ejemplos de los errores que detecta o mitiga.
  - **Generación de código:** traducción de la IR a ensamblador x86, convenciones de llamada y manejo de memoria.
  - **Evaluación experimental:** metodología, entorno experimental y resultados de cada uno de los aspectos evaluados, presentados mediante tablas y gráficos.
  - **Conclusiones y referencias.**

- **Exposición final:** presentación técnica del proyecto, que incluya una demostración del compilador en funcionamiento, del efecto de las optimizaciones y de la detección de errores de seguridad.

Bonificación

El máximo logro del proyecto consistirá en la implementación de una aplicación funcional del compilador desarrollado que permita demostrar, de manera integrada, las distintas fases del proceso de compilación y las principales características del lenguaje implementado. Los grupos que logren desarrollar dicha aplicación podrán obtener hasta **3 puntos adicionales** en el Examen final del curso. La aplicación deberá incluir, como mínimo, los siguientes componentes:

- Editor de código para el lenguaje seleccionado, con resaltado de errores léxicos, sintácticos y semánticos.
- Visualización del AST (*Abstract Syntax Tree*).
- Visualización de la representación intermedia antes y después de aplicar las optimizaciones.
- Generación de código ensamblador x86, con opción para activar o desactivar optimizaciones y mecanismos de seguridad.
- Reporte de las advertencias y errores de seguridad detectados.
- Ejecución o simulación del programa compilado.
- Visualización de resultados de ejecución.